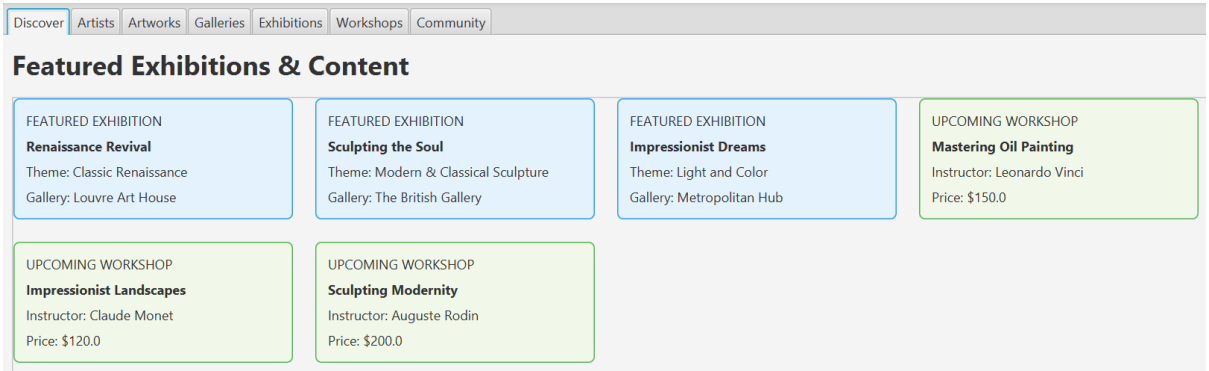
Aube RAMASSAMY – Judia OUAZARI – Isabelle SOMPHONE – Corentin MONSALLIER

Step 1 : Understanding ArtConnect and Defining the Functional Scope

2. Functional analysis

- List the main features you see in the interface (from the user's perspective).



Main features :

- Discover: view the exhibitions and upcoming workshops on a dashboard
- Artists: browse the full list of artists with their name, city, email and birth year
- Artworks: browse the list of artworks with title, artist, type, price and sale status
- Galleries: browse the list of galleries with address and rating
- Exhibitions: view the list of future exhibitions with the title, the host gallery, start date and theme
- Worshops: browse workshops with their title, instructor, date, price and level
- Community: view community members with thier name, email and city

Primary Actors	Secondary actor
- Organizer: creates and manages exhibitions - Member of the community (registered user): browses content, can authenticate, book workshops, write reviews - Viewer (unregistered user): can browse and purchase artworks (must register for that) - Artists (potential instructor): uploads and manages their artworks and profile - Instructors: specialized Artist who leads workshops	Gallery: place that hosts exhibitions Bank: handles payment processing when a viewer buys an artwork

3. Static design

- Read the README file carefully.

- Understand the relationships between the different application classes, as well as the overall architecture.

- Create one (or more) class diagram(s)

Step 2: Conceptual and Logical Modeling of the ArtConnect Database

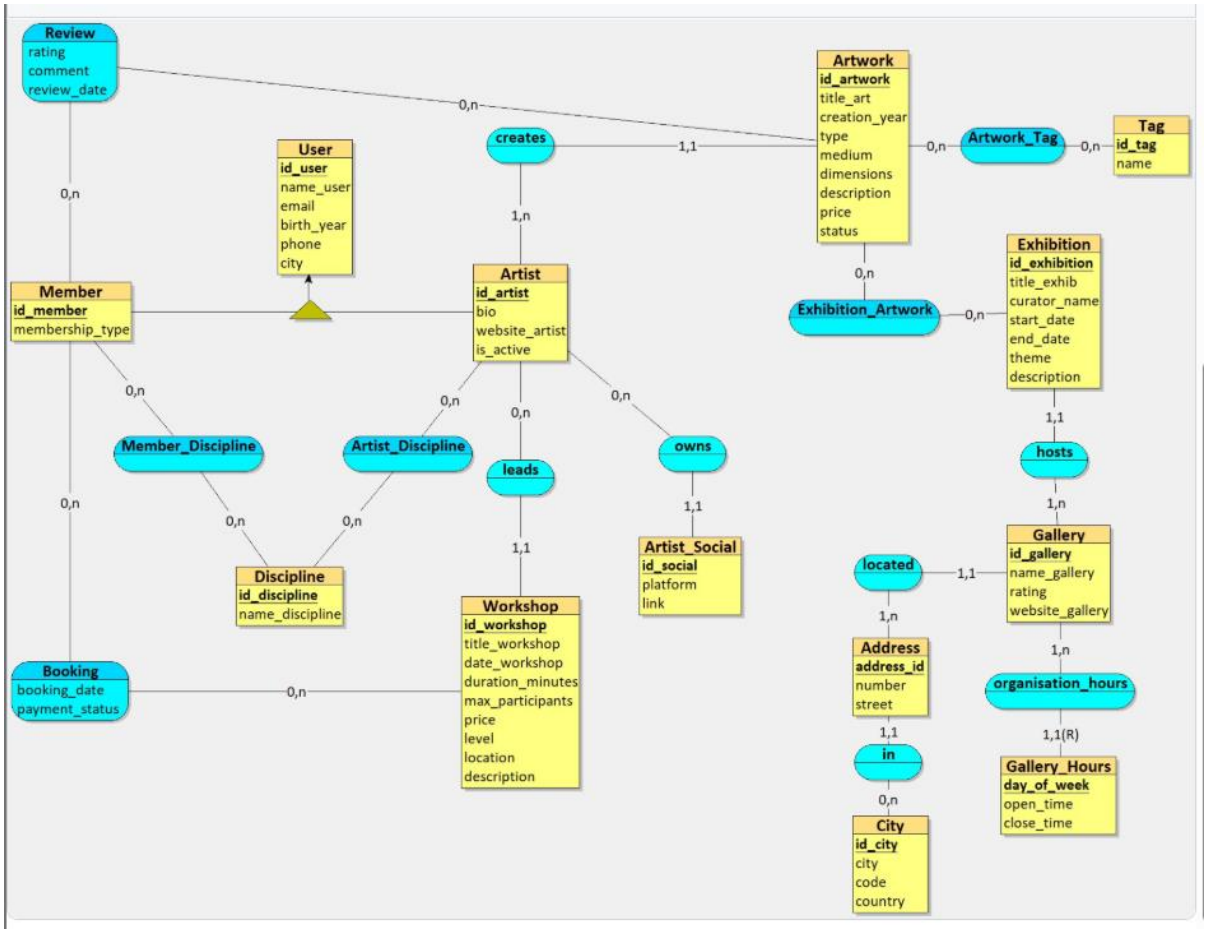
1. Identify entities and attributes

Entities	Attributes	Dependencies	identifiers
User	**Id_user**	Id_user <- Name,	**Id_user**

	Name Email BirthYear Phone City	Email, BirthYear, Phone, City	
Artist	**Id_artist** Bio Website SocialMedia IsActive Disciplines artworks	Id_artist <- Bio, Website, SocialMedia, IsActive, Disciplines, Artworks	**Id_artist**
Discipline	**Id_discipline** name	Id_discipline <- name	**Id_discipline**
Workshop	**Id_workshop** Title Date DurationMinutes MaxParticipants Price Instructor Location Description Level	Id_workshop <- Title, Date, DurationMinutes, MaxParticipants, Price, Instructor, Location, Description, Level, Instructor Date <- DurationMinutes Description <- MaxParticipants, Price, location, level	**Id_workshop**
Community member	**Id_Member** MembershipType FavoriteDiscipline Bookings Review	Id_Member <- MembershipType, FavoriteDiscipline, Bookings, Review	**Id_Member**
Artwork	**Id_Artwork** Title CreationYear Type Medium Dimensions Description Price Status Tags	Id_Artwork <-Title, CreationYear, Type, Medium, Dimensions, Description, Price, Status, Tags Type <- Medium, Dimensions, Description	**Id_Artwork**
Booking	**Id_Booking** BookingDate PaymentStatus Workshop Member	Id_Booking <- BookingDate, PaymentStatus, Workshop, Member	**Id_booking**

Review	**Id_Review** Reviewer Artwork Rating Comment ReviewDate	Id_Review <- Reviewer, Artwork, Rating, Comment, ReviewDate	**Id_Review**
ArtworkTag	**Id_Arttag** Name	Id_Arttag <- Name	**Id_Arttag**
Exhibition	**Id_Exhib** Title StartDate EndDate Description Gallery CuratorName Theme artworks	Id_exhib <- title, StartDate, EndDate, Description, Gallery, CuratorName, Theme, Artworks	**Id_Exhib**
Gallery	**Id_Gallery** Name Address OwnerName OpeningHours ContactPhone Rating Website exhibitions	Id_Gallery <-Name, Address, OwnerName, OpeningHours, ContactPhone, Rating, Website, Exhibitions	**Id_Gallery**

2. Create a Conceptual Data Model (CDM)



3. Create a Logical Data Model (LDM)

User_ = (id_user *INT*, name_user *VARCHAR(50)*, email *VARCHAR(50)*, birth_year *INT*, phone *VARCHAR(20)*, city *VARCHAR(50)*)**;**

Artist = (id_artist *INT*, bio *TEXT*, website_artist *VARCHAR(100)*, is_active *LOGICAL, #id_user*)**;**

Member_ = (id_member *INT*, membership_type *VARCHAR(50), #id_user*)**;**

Discipline = (id_discipline *INT*, name_discipline *VARCHAR(50)*)**;**

Artwork = (id_artwork *INT*, title_art *VARCHAR(100)*, creation_year *INT*, type *VARCHAR(50)*, medium *VARCHAR(50)*, dimensions *VARCHAR(50)*, description *TEXT*, price *DECIMAL(10,2)*, status *ENUM('available', 'sold', 'reserved'), #id_artist*)**;**

Tag = (id_tag *INT*, name *VARCHAR(50)*)**;**

Workshop = (id_workshop *INT*, title_workshop *VARCHAR(100)*, date_workshop *DATETIME*, duration_minutes *INT*, max_participants *INT*, price *DECIMAL(10,2)*, level *VARCHAR(50)*, location *VARCHAR(50)*, description *TEXT, #id_artist*)**;**

Artist_Social = (id_social *INT*, platform *VARCHAR(50)*, link *VARCHAR(200), #id_artist*)**;**

City = (id_city *INT*, city *VARCHAR(50)*, code *INT*, country *VARCHAR(50)*)**;**

Address = (address_id *INT*, number *INT*, street *VARCHAR(50)*, *#id_city*);

Gallery = (id_gallery *INT*, name_gallery *VARCHAR(50)*, rating *INT*, website_gallery *VARCHAR(100)* , ***#address_id*);**

Gallery_Hours = (*#id_gallery*, day_of_week *VARCHAR(20)*, open_time *TIME*, close_time *TIME*);

Exhibition = (id_exhibition *INT*, title_exhib *VARCHAR(100)*, curator_name *VARCHAR(50)*, start_date *DATE*, end_date *DATE*, theme *VARCHAR(50)*, description *TEXT, #id_gallery*);

Artist_Discipline = (*#id_artist, #id_discipline*);

Artwork_Tag = (*#id_artwork, #id_tag*);

Exhibition_Artwork = (*#id_artwork, #id_exhibition*);

Member_Discipline = (*#id_member, #id_discipline*);

Review = (*#id_member, #id_artwork*, rating *INT*, comment *TEXT*, review_date *DATETIME*);

Booking = (*#id_member, #id_workshop*, booking_date *DATETIME*, payment_status *ENUM('pending', 'paid', 'cancelled')*);

4. Create the database schema :

See Database.sql

Step 3 : Database Implementation and Advanced Features

1. Create and populate the database

When giving the files to ChatGPT : “According to these files, please fill each table of the database with at least ten rows.”

2. Views, indexes, and access rights

Goal	View’s code
Artwork by artist	`CREATE VIEW V_Artwork_By_Artist AS` `SELECT` `a.id_artist,` `u.name_user AS artist_name,` `aw.id_artwork,` `aw.title_art,` `aw.creation_year,` `aw.type,`

	aw.price, aw.status FROM Artwork aw JOIN Artist a ON aw.id_artist = a.id_artist JOIN User_ u ON a.id_user = u.id_user ORDER BY a.id_artist, aw.id_artwork;
Artworks by discipline	CREATE VIEW *V_Artworks_By_Discipline* AS SELECT d.id_discipline, d.name_discipline, a.id_artwork, a.title_art FROM Discipline d JOIN Artist_Discipline ad ON d.id_discipline = ad.id_discipline JOIN Artist ar ON ad.id_artist = ar.id_artist JOIN Artwork a ON ar.id_artist = a.id_artist;
Artworks by review	CREATE VIEW *V_Artwork_By_Review* AS SELECT a.id_artwork, a.title_art, a.creation_year, a.type, a.medium, a.dimensions, a.description, a.price, a.status, r.rating, r.comment, r.review_date, r.id_member, u.name_user FROM Review r JOIN Artwork a ON r.id_artwork = a.id_artwork JOIN Member_ m ON r.id_member = m.id_member JOIN User_ u ON m.id_user = u.id_user ORDER BY r.rating;
Average price of the artworks of an artist	CREATE VIEW *V_Avg_Artwork_Price* AS SELECT a.id_artist, u.name_user AS artist_name, *AVG*(aw.price) AS avg_price, *COUNT*(aw.id_artwork) AS artwork_count FROM Artist a JOIN User_ u ON a.id_user = u.id_user JOIN Artwork aw ON aw.id_artist = a.id_artist

	GROUP BY a.id_artist ORDER BY a.id_artist;
Reviews by artwork	CREATE VIEW *V_Review_By_Artwork* AS SELECT a.id_artwork, a.title_art, a.creation_year, a.type, a.medium, a.dimensions, a.description, a.price, a.status, r.rating, r.comment, r.review_date, r.id_member, u.name_user FROM Review r JOIN Artwork a ON r.id_artwork = a.id_artwork JOIN Member_ m ON r.id_member = m.id_member JOIN User_ u ON m.id_user = u.id_user;
Workshop by availability (booking)	CREATE VIEW *V_Workshop_Availability* AS SELECT w.id_workshop, w.title_workshop, w.max_participants, *COUNT*(b.id_member) AS total_bookings, (w.max_participants - *COUNT*(b.id_member)) AS remaining_spots, CASE WHEN *COUNT*(b.id_member) < w.max_participants THEN 'available' ELSE 'full' END AS availability_status FROM Workshop w JOIN Booking b ON w.id_workshop = b.id_workshop AND b.payment_status <> 'cancelled' GROUP BY w.id_workshop, w.title_workshop, w.max_participants;
Members in workshop	CREATE VIEW *V_Workshop_Participants* AS SELECT w.id_workshop, w.title_workshop, m.id_member, u.name_user AS participant_name, b.payment_status,

	b.booking_date FROM Booking b JOIN Workshop w ON b.id_workshop = w.id_workshop JOIN Member_ m ON b.id_member = m.id_member JOIN User_ u ON m.id_user = u.id_user WHERE b.payment_status <> 'cancelled';
Count workshops done already by an artist	CREATE VIEW *V_Artist_Workshop_Count* AS SELECT a.id_artist, u.name_user AS artist_name, *COUNT*(w.id_workshop) AS past_workshops FROM Artist a JOIN User_ u ON a.id_user = u.id_user JOIN Workshop w ON a.id_artist = w.id_artist AND w.date_workshop < NOW() GROUP BY id_artist, u.name_user;
Artist by exhibition	CREATE VIEW *V_Artist_By_Exhibition* AS SELECT e.id_exhibition, e.title_exhib, a.id_artist, u.name_user AS artist_name, *COUNT*(ea.id_artwork) AS artworks_in_exhibition FROM Exhibition e JOIN Exhibition_Artwork ea ON e.id_exhibition = ea.id_exhibition JOIN Artwork aw ON ea.id_artwork = aw.id_artwork JOIN Artist a ON aw.id_artist = a.id_artist JOIN User_ u ON a.id_user = u.id_user GROUP BY e.id_exhibition, a.id_artist ORDER BY e.id_exhibition, a.id_artist;
Available exhibition by gallery	CREATE VIEW *V_Available_Exhibitions_By_Gallery* AS SELECT g.id_gallery, g.name_gallery, e.id_exhibition, e.title_exhib, e.start_date, e.end_date, e.theme FROM Exhibition e JOIN Gallery g ON e.id_gallery = g.id_gallery WHERE CURDATE() BETWEEN e.start_date AND e.end_date;
Artist associated with their social(s)	CREATE VIEW *V_Artist_Social* AS SELECT a.id_artist,

	u.name_user AS artist_name, s.platform, s.link AS social_link FROM Artist a JOIN User_ u ON a.id_user = u.id_user JOIN Artist_Social s ON a.id_artist = s.id_artist;
Regroup info on users	CREATE VIEW *V_User_Info* AS SELECT u.id_user, u.name_user, u.email, u.birth_year, u.phone, u.city, m.membership_type, m.id_member FROM User_ u JOIN Member_ m ON u.id_user = m.id_user;

Goal	Index's code
Artwork	CREATE INDEX *idx_artwork* ON Artwork(id_artwork);
Artists	CREATE INDEX *idx_artist* ON Artist(id_artist);
Member	CREATE INDEX *idx_member* ON Member_(id_member);

3. Triggers and stored programs

Goal	Trigger's code
Rating between 1 and 5	DELIMITER // CREATE TRIGGER *rating_range* BEFORE INSERT ON review FOR EACH ROW BEGIN IF NEW.rating > 5 OR NEW.rating < 1 THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: rating must be between 1 and 5!'; END IF; END; // DELIMITER ;
End date >= Start date	DELIMITER // CREATE TRIGGER *start_end_date* BEFORE INSERT ON Exhibition FOR EACH ROW

	BEGIN IF NEW.end_date < NEW.start_date THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: the end date must not be before the start date!'; END IF; END; // DELIMITER ;
Booking possible only if max_participant not reach	DELIMITER // CREATE TRIGGER *available_booking* BEFORE INSERT ON Booking FOR EACH ROW BEGIN DECLARE max_capacity INT; DECLARE current_bookings INT; *-- max capacity of the workshop* SELECT max_participants INTO max_capacity FROM Workshop WHERE id_workshop = NEW.id_workshop; *-- current booked participants* SELECT *COUNT*(*) INTO current_bookings FROM Booking WHERE id_workshop = NEW.id_workshop AND payment_status != 'cancelled'; IF current_bookings >= max_capacity THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Error: no available places left for this workshop!'; END IF; END; // DELIMITER ;

Goal	Function's code
Return the number of participant for a workshop	DELIMITER // CREATE FUNCTION *F_Nb_Participants_Count*(p_id_workshop INT) RETURNS INT DETERMINISTIC BEGIN DECLARE f_total INT; SELECT *COUNT*(*) INTO f_total FROM Booking WHERE id_workshop = p_id_workshop

	`` AND payment_status != 'cancelled'; RETURN f_total; END // DELIMITER ; ```

Goal	Stored procedure's code
Creation of a workshop	``` DELIMITER // CREATE PROCEDURE Create_Workshop (IN p_title VARCHAR(100), IN p_date DATETIME, IN p_duration INT, IN p_max_participants INT, IN p_price DECIMAL(10,2), IN p_level VARCHAR(50), IN p_location VARCHAR(50), IN p_description TEXT, IN p_id_artist INT) BEGIN INSERT INTO Workshop (title_workshop, date_workshop, duration_minutes, max_participants, price, level, location, description, id_artist) VALUES (p_title, p_date, p_duration, p_max_participants, p_price, p_level, p_location, p_description, p_id_artist); END // DELIMITER ; ```
Creation of an exibition	``` DELIMITER // CREATE PROCEDURE Create_Exhibition (IN p_title_exhib VARCHAR(100), ```

	`` IN p_curator_name VARCHAR(50), IN p_start_date DATE, IN p_end_date DATE, IN p_theme VARCHAR(50), IN p_description TEXT, IN p_id_gallery INT) BEGIN INSERT INTO Exhibition (title_exhib, curator_name, start_date, end_date, theme, description, id_gallery) VALUES (p_title_exhib, p_curator_name, p_start_date, p_end_date, p_theme, p_description, p_id_gallery); END // DELIMITER ; ```
Count remaining spots in workshop	``` DELIMITER // CREATE PROCEDURE Check_Workshop_Capacity(IN p_id_workshop INT) BEGIN DECLARE p_max INT; DECLARE p_current INT; -- max capacity of the workshop SELECT COUNT(*) INTO p_current FROM Booking WHERE id_workshop = p_id_workshop AND payment_status != 'cancelled'; -- current booked participants SELECT COUNT(*) INTO p_current FROM Booking WHERE id_workshop = p_id_workshop AND payment_status != 'cancelled'; -- remaining spots SELECT ```

	p_max AS Capacity, p_current AS Booked, (p_max - p_current) AS Spots_Left; END // DELIMITER ;
Register a member in a workshop	DELIMITER // CREATE PROCEDURE Register_Member_To_Workshop(IN p_id_member INT, IN p_id_workshop INT) BEGIN DECLARE p_current_participants INT; DECLARE p_max_places INT; -- current booked participants SELECT COUNT(*) INTO p_current_participants FROM Booking WHERE id_workshop = p_id_workshop AND payment_status != 'cancelled'; -- maximum participants SELECT max_participants INTO p_max_places FROM Workshop WHERE id_workshop = p_id_workshop; IF p_current_participants < p_max_places THEN INSERT INTO Booking(id_member, id_workshop, booking_date, payment_status) VALUES(p_id_member, p_id_workshop, NOW(), 'pending'); ELSE SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Operation Error: no available places left for this workshop!'; END IF; END // DELIMITER ;

4. Transactions

Goal	Transaction's code
Register a user to multiple workshops	

```
DELIMITER //
CREATE PROCEDURE Register_To_Multiple_Workshops(
    IN p_id_member INT,
    IN p_id_workshop1 INT,
    IN p_id_workshop2 INT
)
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
    END;
    START TRANSACTION;
    CALL Register_Member_To_Workshop(p_id_member,
p_id_workshop1);
    CALL Register_Member_To_Workshop(p_id_member,
p_id_workshop2);
    COMMIT;
END //
DELIMITER ;
```

Goal	Transaction's test code
This code was used to test whether the transaction works well or not	

```
-- ============================
-- TEST 1: SUCCESSFUL TRANSACTION
-- ============================
-- 1. Check bookings for member 5
SELECT * FROM Booking WHERE id_member = 5;
-- 2. Execute the workshop registration
CALL Register_To_Multiple_Workshops(5, 1, 2);
-- 3. Verify that bookings exist
SELECT * FROM Booking WHERE id_member = 5;



-- ============================
-- TEST 2: FAILURE & ROLLBACK
-- ============================
-- 1. Check bookings for member 6
SELECT * FROM Booking WHERE id_member = 6;
-- 2. Execute the workshop registration
CALL Register_To_Multiple_Workshops(6, 3, 999);
-- 3. Verify that bookings don't exist
SELECT * FROM Booking WHERE id_member = 6;
```